GABRIEL POPA

Senior Software Engineer - React • Python(Django/ FastAPI) • AWS
Cluj, Romania | +40 764309991 | gabrielpdev@outlook.com | https://www.linkedin.com/in/gabriel-popa-12890b414/

ABOUT ME

Senior Software Engineer with 10+ years of experience building reliable web platforms with **Python** and **React**, covering backend services, API design, data workflows, and responsive frontend delivery across product teams. Strong hands-on background with **Python 2.7–3.12**, **React 0.14–18**, **Django 1.x–4.x**, **FastAPI**, **Flask**, **JavaScript/TypeScript**, **PostgreSQL**, **Redis**, **Docker**, and cloud deployment patterns, with a practical record of modernizing legacy code, fixing production issues, improving release safety, and delivering maintainable features that balance speed, stability, and long-term system health.

SKILLS

- **Frontend: React 0.14–18**, **JavaScript ES5–ES2023**, **TypeScript 3.x–5.x**, HTML5, CSS3, Sass, Responsive UI, Component Design, Hooks, Context API, Redux, React Query, Form Handling
- **Backend: Python 2.7–3.12**, **Django 1.x–4.x**, **FastAPI 0.6+**, **Flask 0.10+**, REST API Design, Authentication, Background Jobs, Data Processing, Service Integration, Validation, Error Handling
- **Data & Messaging: PostgreSQL**, MySQL, SQLite, **Redis**, Celery, RabbitMQ, Query Optimization, Caching, Reporting Workflows, ETL Support
- **Cloud & DevOps:** AWS, Docker, Linux, Nginx, CI/CD, GitHub Actions, Jenkins, Deployment Automation, Environment Configuration, Monitoring Support
- **Observability / Quality / Security:** OpenTelemetry basics, Prometheus/Grafana basics, Sentry, structured logging, distributed tracing basics, rate limiting, OWASP security practices, **Pytest**, **Jest**, **React Testing Library**, **Supertest**, **Cypress**, **Playwright**, unit testing, integration testing, contract testing, end-to-end testing, API testing, and error monitoring

PROFESSIONAL EXPERIENCE

Senior Software Engineer

Soft Build | Bucharest, Romania *(Jan 2024 – May 2026)*

- Led a platform modernization effort where several operational workflows had become difficult to extend, then redesigned key modules with **Python 3.11–3.12**, **FastAPI**, and **React 18**, which improved delivery speed, reduced duplicated logic, and made future feature work safer across teams.
- Owned a recurring frontend reliability problem in complex form flows, then introduced shared components with **React Hooks**, **Context API**, and **TypeScript 5.x**, which removed inconsistent local state behavior and reduced validation-related regressions in high-usage screens.
- Took responsibility for unstable API behavior that caused frontend crashes during partial backend failures, then implemented stricter request validation, safer error contracts, and better response consistency in **FastAPI**, which improved resilience and simplified frontend integration work.
- Addressed slow dashboard performance affecting day-to-day reporting by tuning **PostgreSQL** queries, reducing unnecessary joins, and applying selective **Redis** caching, which stabilized response times and improved the user experience on filter-heavy pages.
- Resolved a production issue where simultaneous edits occasionally overwrote recent changes by introducing optimistic concurrency checks, clearer conflict responses, and safer retry handling, which prevented silent data loss and gave users a more predictable recovery path.
- Improved incident investigation on a system where frontend and backend failures were difficult to correlate, then added structured logging, **Sentry**, and basic **OpenTelemetry** instrumentation, which shortened debugging time by **35%** and made production issues easier to isolate.
- Owned visibility gaps across background jobs and request flows, then introduced request correlation and basic distributed tracing practices between APIs and **Celery** workers, which helped the team find latency bottlenecks and better understand failure paths.

- Fixed repeated timeout problems in export and synchronization workflows by moving long-running work into **Celery** with **Redis**, then exposing job progress more clearly in the UI, which reduced failed requests and improved operational confidence for users.
- Strengthened weak security controls around sensitive endpoints by applying **OWASP**-aligned validation, safer authentication handling, and practical rate limiting, which lowered abuse risk without disrupting normal workflows for trusted users.
- Improved release confidence in areas with repeated regressions by expanding automated coverage with **Pytest**, **Jest**, **React Testing Library**, and **Supertest**, which made refactoring safer and reduced defects in permission, validation, and workflow-heavy modules by **30%**.
- Took ownership of inconsistent deployment behavior between environments, then tightened **GitHub Actions** pipelines with stronger validation and release checks, which reduced configuration drift and improved the predictability of production rollouts.
- Investigated a memory growth problem in a Python batch process by analyzing object retention patterns and execution flow, then refactored the job into smaller chunks with safer cleanup logic, which stabilized worker performance during large processing runs.
- Improved operational awareness where teams lacked useful health signals by exposing basic metrics and alert-friendly telemetry with **Prometheus/Grafana** basics, which gave **40%** earlier visibility into queue delays, error spikes, and degraded endpoints.
- Handled a runtime upgrade that had been postponed because of dependency risk, then migrated older Python services in stages with targeted refactors and stronger tests, which reduced rollout friction and moved the application onto a more supportable baseline.
- Worked closely with QA, product, and engineering stakeholders during fast-changing delivery cycles, translating unclear requirements into smaller safe releases that balanced feature progress with testing discipline, observability, and long-term maintainability.

Software Engineer

Next Apps | Ghent, Belgium *(Nov2020 – Nov 2023)*

- Joined a codebase that mixed old and new implementation styles, then delivered product features with **Python 3.8–3.11**, **Django**, **FastAPI**, **React 16–18**, and **TypeScript 4.x–5.x**, which improved consistency across the stack while keeping delivery practical and maintainable.
- Took on fragile frontend areas that were difficult to debug and extend, then migrated selected class-based components toward **Hooks** and reusable patterns, which reduced lifecycle-related defects and made new UI development more predictable.
- Supported business-critical integrations where third-party instability caused user-facing failures, then added retry handling, payload normalization, and structured logging, which improved reliability and made upstream issues easier to diagnose.
- Investigated inconsistent dashboard totals that were confusing users and support teams, then tightened **Redis** invalidation logic and backend update sequencing, which improved data consistency and reduced stale-cache related reporting errors.
- Improved delivery speed on repetitive UI work by building reusable forms, tables, and validation patterns in **React** and **TypeScript**, which reduced copy-paste implementation by **35%** and made feature development faster across several modules.
- Strengthened a weak testing story around API and backend refactors by introducing more disciplined **Pytest** coverage and service-level verification, which increased confidence when updating permissions, validation rules, and processing logic.
- Resolved stuck-task and duplicate-processing issues in asynchronous workflows by reviewing retry strategy, timeout behavior, and cleanup in **Celery** and **Redis**, which improved job reliability and reduced operational confusion.
- Improved production visibility where exceptions were getting lost in logs by adding **Sentry** and cleaner exception mapping across backend and frontend layers, which made real failures easier to see and prioritize **40% faster** before they escalated.
- Helped standardize local and deployment environments with **Docker** and cleaner configuration handling, which reduced onboarding friction and made production-like issues easier for developers to reproduce and fix earlier.

- Identified security gaps on exposed endpoints during feature expansion, then applied stronger request validation, authentication checks, and basic rate limiting, which improved API resilience against malformed or abusive traffic.
- Reduced frontend regression risk in interactive user journeys by expanding coverage with **Jest**, **React Testing Library**, and targeted end-to-end checks, which improved reliability in multi-step forms and asynchronous UI states by **30%**.

Software Developer

Datamart | Stockholm, Sweden *(Feb 2016 – Sep 2020)*

- Worked on business applications built with **Python 3.5–3.8**, **Django**, **Flask**, and **React 15–16**, helping the team evolve server-rendered workflows into more maintainable full-stack solutions without pushing architecture changes beyond what the platform could support.
- Supported internal dashboards and reporting tools where usability problems slowed operational work, then improved backend behavior and key UI flows, which made approval, search, and export processes more reliable for daily users.
- Reduced repeated frontend rework in similar admin screens by introducing reusable table, filter, and modal-edit patterns, which improved consistency by **30%** and lowered the risk of small UI regressions during ongoing product changes.
- Helped modernize interaction-heavy areas by moving selected screens from template-heavy rendering toward component-based **React** views, which simplified frontend behavior and made interactive logic easier to maintain over time.
- Solved a persistent reporting bug caused by timezone mismatches between browser input and backend processing, then normalized parsing and storage rules, which improved data consistency and reduced confusion in exported and scheduled reports.
- Improved slow reporting workflows by rewriting inefficient **PostgreSQL** queries, reducing repeated reads, and adding missing indexes, which cut unnecessary load by **35%** and made high-use administrative screens noticeably more responsive.
- Built and maintained internal **REST**-style endpoints where frontend and backend concerns had started to blur, then improved contract clarity and separation of responsibilities, which made the application easier to extend incrementally.
- Reduced operational strain from long-running imports and scheduled tasks by moving heavy work into background processing, which kept normal request handling more stable and reduced disruption during periods of high administrative activity by **25%**.
- Improved deployment safety in a release process that relied too heavily on manual steps by supporting **Jenkins**, deployment scripts, and environment validation, which lowered packaging mistakes by **30%** and made releases more repeatable.
- Made production defects easier to troubleshoot by improving logging discipline and surfacing clearer failure information in batch and reporting flows, which reduced time lost to vague user reports and incomplete reproduction details.
- Strengthened confidence around backend changes by adding focused **Pytest** unit and integration coverage in validation, transformation, and scheduled processing areas, which reduced regression risk during routine maintenance and refactoring.

Junior Software Developer

Berg Software | Timișoara, Romania *(Sep2014 – Mar 2016)*

- Contributed to a legacy business application built with **Python 2.7–3.5**, **Django**, **JavaScript**, and early **React 0.14–15**, helping the team deliver small but important changes without disrupting tightly coupled production workflows.
- Maintained server-rendered pages, backend forms, and validation logic in modules where stable behavior mattered more than large rewrites, which helped keep day-to-day user operations reliable during ongoing feature updates.
- Added small **React** widgets to selected pages for interactive behaviors such as inline actions, dynamic form updates, and lightweight status displays, while keeping the implementation compatible with the existing template-driven structure.
- Fixed a recurring login issue that caused unexpected session drops by helping align cookie settings, timeout behavior, and request handling between browser and backend flows, which improved sign-in stability for regular users.

- Improved form handling in several admin screens by tightening backend validation, cleaning inconsistent error messages, and correcting edge-case input behavior, which reduced failed submissions by **25%** and made user correction steps easier to follow.
- Assisted with defect investigation by reproducing bugs from support notes, logs, and tester feedback, then preparing small targeted fixes that were easier for senior engineers to review, verify, and release safely.
- Helped improve performance on slower administrative pages by reducing unnecessary database queries and simplifying backend data preparation, which made frequently used maintenance screens more responsive without major architectural changes.
- Supported senior engineers during framework and dependency update work by checking compatibility issues, fixing small deprecated code paths, and verifying affected screens, which reduced upgrade-related regressions in fragile areas.
- Built small reusable helper functions and shared UI utilities for repeated behaviors across forms and admin pages, which reduced copy-paste implementation and improved consistency when similar changes were needed later.
- Added basic tests and extra logging in selected workflows while assisting with QA verification and release checks, which helped the team catch common regressions earlier and made support-side debugging more straightforward.

EDUCATION

Bachelor's Degree in Computer Science

University of Bucharest | (*2010 – 2014)*

- Focused on practical software engineering fundamentals that translated directly into building reliable web systems and data-driven applications.

CERITIFICATIONS

AWS Certified Developer – Associate — 2023

- Demonstrated practical capability in developing and maintaining **AWS**-based applications, with focus on deployment, service integration, monitoring, and secure cloud operations.

PCAP – Certified Associate in Python Programming — 2022

- Demonstrated strong foundation in **Python** programming, including core language features, modular design, debugging, and problem-solving for backend-oriented applications.

Scrum Fundamentals Certified (SFC) — 2021

- Demonstrated working knowledge of agile principles, sprint-based execution, team collaboration, and structured delivery practices within software development environments.